

Speaker notes

A script for the talk, slide by slide, in the order of deck. json. Written 2026-10-01 for the session on October 2, 2026. The slides' own notes (press N in the deck) are the presenter's instructions: what to switch to and what not to give away. This is what to say.

To print: SPEAKER_NOTES.pdf, US Letter, large type, each slide kept on one page. It is built from this file by `pnpm notes (docs/notes.mjs)`, so rebuild and commit it whenever this file changes.

Every segment is written to come in under its slot in PRESENTATION.md *Timing*. The **spoken** times assume 110 words a minute, a slow, paused pace, and come from a word count of the *Say* text (2026-10-01). They leave the rest of each slot for the room: joining, questions, laughter, and slides changing. The script is kept short, so you read a line and look up.

Clock	Segment	Slot	Spoken	Slides
0:00	Welcome and join	3 min	≈1:05	cover, agenda, join
0:03	Core principles	10 min	≈6:25	threeways, flow, constraint, feedback, learning, dora
0:13	How to play	2 min	≈1:15	question, howto
0:15	Play	15 min	≈0:20	play
0:30	Leaderboard	3 min	≈0:15 + the room	leaderboard
0:33	Debrief	9 min	≈5:10	lessons through rubber (11 slides)
0:42	Monday and questions	3 min	≈0:50	checklist, close, sources

The principles block carries the two things this talk must land, the Three Ways and the DORA keys, so it gets the most words. The debrief is the tight one: 11 slides in 9 minutes. If you run long there, the cases (`replit`, `kiro`, `amazon`, `dex`) can each drop to their one-line *Short* version.

0:00 Welcome and join

3 min, ≈1:05 spoken

COVER

≈0:30

Good morning. Before anything else, get out a laptop or a phone, because in two minutes you'll be playing a game. The title is the argument: code is cheap now, attention isn't. I'm not going to ask you to believe it. I'd rather you find it in the game.

AGENDA

≈0:20

Here's the plan. I talk for about ten minutes, you play for fifteen, and the last fifteen minutes are about what you just did. A third of this session is hands-on.

JOIN

≈0:20, then wait
for the room

Switch to the presenter tab with the QR code.

Scan the code or type the link, and enter the name you want on the leaderboard. It opens paused, with a short walkthrough. Click through it, but don't press Start yet. We'll all start together.

Since 2026-10-05 the join slide shows free play, not a session: the talk is over. To give it again, start a session at ?present and put its code, link and QR code back on the slide first.

0:03 Core principles

10 min, ≈6:25 spoken

The two things to land are the Three Ways (with *The Phoenix Project*) and the DORA keys, so threeways and dora get the most time. About 3:30 is left in the slot for the room and for going off script.

THREEWAYS

≈2:10

DevOps has a hundred definitions. The one I find most useful is the Three Ways.

The **First Way is flow**: work moving left to right, from idea to production, fast and in small batches. The **Second Way is feedback**: information moving right to left, so problems get fixed where they started instead of downstream. The **Third Way is continual learning**: improving the system is part of the job, not something you get to after the job.

The slide credits *The DevOps Handbook*, but I first met the Three Ways in *The Phoenix Project*, and I really enjoyed it. It's a **novel** about IT operations. Which is a strange thing to exist. A thriller where the stakes are a release, and the villain is unplanned work. Bill runs IT operations at a struggling company called Parts Unlimited, he's handed a failing project called Phoenix, and every week is a new fire. A mentor walks him through a factory floor and teaches him the Three Ways.

It's strange, but it works, because DevOps problems aren't really technical. They're people, queues and handoffs, and a story shows you that better than a framework does. It's on purpose, too: it's modeled on *The Goal*, a 1984 novel about a factory, which is where the bottleneck idea comes from.

Hold on to one character: Brent, the engineer every piece of work has to go through. Remember him.

FLOW

≈0:45

The First Way has one law behind it. Little's Law: lead time equals work in progress divided by throughput. If you can't finish faster, the only way to make work take less time is to have less of it in flight. Software adds a twist a factory doesn't have: unfinished work decays. Every deploy moves trunk, and every open branch drifts further behind. There's a right amount of work in progress. The game will show you where it is.

CONSTRAINT

≈0:55

Every system has one bottleneck. An hour saved there is an hour saved for the whole system. An hour saved anywhere else is an illusion. That idea is from *The Goal*, and *The Phoenix Project* borrows it. In the novel, the bottleneck is Brent. Not a server, a person: everything waits on his attention. Keep that in mind for later. Find the constraint, get the most out of it, make everything else serve it, then add capacity, and look again, because it moves. The game names its bottleneck after day 10. Watch for it.

FEEDBACK

≈0:25

The Second Way: find problems while they're cheap. Tests fast enough that people wait for them, the authority to stop the line, and telemetry that tells you before your customers do. Code review is a feedback loop too. Park that; it comes back.

LEARNING

≈0:35

The Third Way: improving the work is the work. Reserve time for it, run blameless reviews, and run small experiments instead of big bets. The last year has been one big experiment in how software gets built. The teams that did well treated it as an experiment. The teams that got hurt treated it as a purchase.

DORA

≈1:50

So how do you know any of this works? DORA: years of research, written up in *Accelerate*. Four keys. Two for **throughput**: lead time for changes, and how often you deploy. Two for **stability**: how often a change fails, and how fast you restore service when one does.

Why does this matter? Look at what we used to measure. Lines of code, story points, commits, utilization: all **activity**, and all about individuals. DORA measures **outcomes**: did a change reach users, and did it hold up. It measures the whole system, from commit to production, not one person.

And the old world measured speed and stability **separately**. Developers were rewarded for shipping, operations for uptime, and the two fought. That's the war at the start of *The Phoenix Project*. DORA puts both on the same team and found they **move together**. The best teams don't trade one for the other; they get both from the same practices.

Last difference: the old metric was time *between* failures. DORA asks how fast you *recover*. Assume things break, and get good at fixing them.

Remember these four. When agents arrived, individual output went way up, and these four mostly didn't move.

0:13 How to play

2 min, ≈1:15 spoken

QUESTION

≈0:15

So here's the question for today. What happens to all of this when writing code becomes nearly free? I won't answer it. You will.

HOWTO

≈1:00

You run a delivery line: nine people, five stations, fifty days. Work arrives in the backlog and flows left to right. Teal is healthy, amber is drifting behind trunk, rust is stale. Four levers. WIP limits per station. Move a person, though it takes them two shifts to learn the station. Add an AI agent: free capacity, but it draws on your attention meter, because someone has to read what it writes. And stale work: rebase it, abandon it, or ship it as it is. You keep up if ten or fewer items are unfinished at the end. The score counts each item at the quality it shipped at.

0:15 Play

15 min, ≈0:20 spoken

PLAY

≈0:20

Press Start. Pausing is free, so think as long as you like. Try 1× until day 10, then 2×. If you finish early, play again; your best run counts.

Start a visible 15-minute timer. Walk the room. Listen for "my agents aren't doing anything" and "everything's gone red", and write down the words. Warn at 3 minutes and at 1.

0:30 Leaderboard

3 min, ≈0:15 spoken, the rest is the room

LEADERBOARD

≈0:15

Switch to the presenter tab.

Every score here was replayed on the server before it was ranked. Hands up if you kept up. ... Let's look at three runs.

Open the winner, a middle run and one near the bottom. Ask each player one question: "What did you change, and when?"

LESSONS

≈0:45

Three patterns, and you just described all of them. **One: less in flight shipped more.** Opening the limits felt productive. It made more work go stale. **Two: the bottleneck moved to Review.** Anything added elsewhere queued in front of it. That's Brent. **Three: agents only paid off once review could keep up.** Idle agents were a review problem, not a capacity problem. Moving a person to Review is what let them work. Who saw "my agents aren't doing anything"? (*Take one answer.*)

ATTENTION

≈0:30

That's the answer to the question. When code gets nearly free, the Three Ways still hold. The constraint moves: from typing to the thinking on either side of it. Money buys agents. It can't buy another hour of careful reading. Let's check whether the real world agrees with a game.

CHANGED

≈0:30

Four changes I've seen this year, and each one is a principle from earlier. **Planning** got more valuable: agents amplify a vague spec. **Review** is the constraint, live. **WIP** exploded, because starting work is now one click: Little's Law. And **agent review** rubber-stamps: once a metric is cheap to satisfy, it will be.

EVIDENCE

≈0:35

The research. Faros AI: teams with heavy AI use merged twice the pull requests, review time nearly doubled, and DORA metrics didn't improve. CodeRabbit: AI-written changes carried about 1.7 times the issues. METR: experienced developers were 19% slower with early-2025 tools and believed they were faster. And DORA's own 2025 report: AI amplifies what a team already is.

REPLIT

≈0:20

July 2025. Replit's agent wiped a production database during an explicit code freeze. "Don't touch production" was a sentence in a chat, not a control. Replit's fix: separate dev from prod, and a planning mode.

Short: An instruction isn't a control. The fix was a planning mode.

KIRO

≈0:25

December 2025, AWS. An agent fixing a production issue decided to delete and recreate the environment: a reported 13-hour outage. It had inherited an engineer's broad permissions. Amazon called it an access error, and added mandatory peer review.

Short: The agent went around two-person approval. The fix was more review.

AMAZON

≈0:20

March 2026, Amazon retail. Outages linked to AI-assisted changes. The response: a 90-day code safety reset, and senior sign-off on AI-assisted changes. More senior attention, spent at review.

Don't quote a lost-orders figure (see the slide's own note).

DEX

≈0:35

And one team that stopped reading the code. Dex Horthy's team let agents handle tickets, crashes and review. About three months in, production broke, and no agent could find the cause. It took days of people reading to find a design problem, then three weeks to relearn a codebase nobody had read. In his words: "None of these things have intuition for software architecture."

Short: They stopped reading the code, and paid three weeks to relearn it.

MINE

≈0:40, your own words

Say, from the slide: Planning starts as a team conversation, then a plan with the AI, written to a markdown file for anything large. In review, I skim first: what changed and how risky is it, before going deep with the AI. AI review sees the trees, not the forest. And my miss: spending time, tokens and meetings on things the AI flagged that weren't the goal. Don't let it set your priorities.

ENDS

≈0:25

So here's the model. People at both ends: plan and review. Agents in the middle, held to a WIP limit set by your review capacity, not your budget. Every case we just saw was fixed by adding attention at one of the two ends.

RUBBER

≈0:25

Agent review is worth having. Let it clear the mechanical fixes: bugs, edge cases, style. Everything it needs is in the diff. What it can't judge is whether the change is right, because the intent isn't in the diff. That's a person's call.

0:42 Monday and questions

3 min, ≈0:50 spoken

CHECKLIST

≈0:35

Three things for Monday. **One:** budget attention like capacity. Cap agent work at what your reviewers can actually read. **Two:** people at plan and review. Written intent before work starts, a named person who owns every merge, and no agent-only path to production. **Three:** measure true quality. The DORA stability keys, rework, escaped defects. PR counts and approvals are this decade's lines of code.

CLOSE

≈0:15

Code is cheap. Attention isn't. Spend it where the constraint is. The game stays up; play it any time. Questions?

SOURCES

■ *Leave it up during questions.*

If someone asks

- **"Is *The Phoenix Project* worth reading?"** Yes, and it's a fast read because it's a novel. Gene Kim, Kevin Behr and George Spafford, 2013. The sequel, *The Unicorn Project* (2019), tells the same story from a developer's side. *The DevOps Handbook* is the how-to, and *Accelerate* is the evidence.
- **"Aren't the four keys out of date?"** DORA has kept refining them since *Accelerate*: the names and the count have shifted, and it has added measures such as reliability and rework. Check dora.dev before quoting

the current list. The idea is unchanged: throughput and stability, measured together, at the level of the system.

- **"Doesn't *Accelerate* say change approval boards make things worse? You're arguing for more review."**
It found that heavyweight approval by people far from the change hurt throughput without making changes safer, and that lightweight review by peers who understand the change didn't. That's the distinction here: the review that works is close to the change, by someone who knows why it exists, not a sign-off queue.
- **"What are the game's numbers?"** Don't quote them as findings. The game shows the ideas, it isn't a study. The measurements are in the repo's docs, starting at `HIRING_AND_ATTENTION §10`.